

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

ASIGNATURA: APLICACIONES WEB

CURSO: 5TO SOFTWARE “A”

**Sistema de Gestión de Recursos Operativos,
Administrativos y de Seguridad para
Cooperativas de Transporte (SGROAS)**

Especificación de Requisitos de Software (SRS) — v1.1.0

<https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb>

DOI Software: 10.5281/zenodo.22522109 | DOI Dataset: 10.5281/zenodo.21973297

Docente-director: Guerrero Ulloa Gleiston Cicerón

Equipo: ACERS

CASTRO ESPINOZA KEVIN MOISÉS

ORCID: 0009-0004-4935-0439

ESCUDERO PLAZA MARÍA DEL

ROSARIO

ORCID: 0009-0007-3212-9924

TEJADA BAJAÑA LUIS ALEJANDRO

ORCID: 0009-0006-2080-1798

Periodo académico:

2026–2027 PPA

Fecha de emisión:

09 de septiembre de 2026

Especificación de Requisitos de Software (SRS) — SGROAS

Versión: v1.1.0 (corregida tras revisión del docente-director, 09-sep-2026)

Conforme a: ISO/IEC/IEEE 29148:2018 (estructura de SRS), INCOSE Guide to Writing Requirements v4 (calidad de requisitos), criterios INVEST de Cohn (historias de usuario), plantilla de Cockburn (casos de uso).

Estado: documento corregido según 16 observaciones del docente-director. Se agregaron 24 requisitos nuevos, se corrigieron conteos, endpoints, estados y cifras inconsistentes.

Índice

1. Introducción	2
1.1. Propósito	2
1.2. Alcance	2
1.3. Definiciones, acrónimos y abreviaturas	2
1.4. Referencias	3
1.5. Resumen del documento	3
2. Descripción global	4
2.1. Perspectiva del producto	4
2.2. Funciones del producto (resumen)	4
2.3. Características de los usuarios	4
2.4. Restricciones	5
2.5. Supuestos y dependencias	5
3. Requisitos específicos	6
3.1. Vocabulario de estados de verificación	6
3.2. Vocabulario de métodos de verificación	6
3.3. Matriz de permisos por rol y operación	6
3.4. Requisitos funcionales (REQ-F)	7
3.4.1. REQ-F-001 — Inicio de sesión	7
3.4.2. REQ-F-002 — Consulta de usuario autenticado	7
3.4.3. REQ-F-003 — Renovación de token	8

3.4.4. REQ-F-004 — Cierre de sesión con revocación	8
3.4.5. REQ-F-005 — Verificación de correo electrónico	8
3.4.6. REQ-F-006 — Reenvío de código de verificación	9
3.4.7. REQ-F-007 — Solicitud de restablecimiento de contraseña	9
3.4.8. REQ-F-008 — Restablecimiento efectivo de contraseña	10
3.4.9. REQ-F-009 — Control de acceso por rol (RBAC)	10
3.4.10. REQ-F-010 — Listado de conductores	10
3.4.11. REQ-F-011 — Consulta de conductor por id	11
3.4.12. REQ-F-012 — Alta de conductor	11
3.4.13. REQ-F-013 — Actualización de conductor	11
3.4.14. REQ-F-014 — Eliminación lógica de conductor	11
3.4.15. REQ-F-015 a REQ-F-019 — CRUD de usuarios	12
3.4.16. REQ-F-020 a REQ-F-024 — CRUD de rutas	12
3.4.17. REQ-F-025 a REQ-F-029 — CRUD de vehículos	13
3.4.18. REQ-F-030 a REQ-F-034 — CRUD de asignaciones	13
3.4.19. REQ-F-035 a REQ-F-039 — CRUD de incidentes	14
3.4.20. REQ-F-040 a REQ-F-045 — Reportes funcionales	14
3.4.21. REQ-F-046 a REQ-F-050 — CRUD de programaciones (ABD)	15
3.4.22. REQ-F-050 a REQ-F-054 — CRUD de unidades (ABD)	15
3.4.23. REQ-F-055 a REQ-F-059 — CRUD de rutas ABD	16
3.4.24. REQ-F-060 a REQ-F-063 — CRUD de incidentes ABD	16
3.4.25. REQ-F-064 — Consulta de alertas ABD	17
3.4.26. REQ-F-065 — Consulta de catálogos ABD	17
3.4.27. REQ-F-066 — Consulta de conductores ABD	17
3.4.28. REQ-F-067 a REQ-F-071 — Reportes ABD	17
3.4.29. REQ-F-074 — Política de contraseñas	18
3.4.30. REQ-F-075 — Protección de datos personales	18
3.4.31. REQ-F-076 — Respaldo de la base de datos	19
3.4.32. REQ-F-077 — Accesibilidad y usabilidad	19

4. Requisitos no funcionales (REQ-NF) 20

4.0.1. REQ-NF-001 — Cabeceras de seguridad HTTP	20
4.0.2. REQ-NF-002 — Cifrado en tránsito	20
4.0.3. REQ-NF-003 — Rendimiento del listado	20
4.0.4. REQ-NF-004 — Protección contra inyección SQL	21
4.0.5. REQ-NF-005 — Limitación de intentos de login	21
4.0.6. REQ-NF-006 — Cobertura de pruebas	21
4.0.7. REQ-NF-007 — Manejo uniforme de errores	22
4.0.8. REQ-NF-008 — Arquitectura en capas	22
4.0.9. REQ-NF-009 — Estrategia de acceso a datos	22
4.0.10. REQ-NF-010 — Disponibilidad de Redis	22
5. Trazabilidad (resumen)	24
5.1. Trazabilidad histórica	24
6. Modelo de datos (referencia)	25
7. Interfaces de usuario (referencia)	26
8. Observaciones e información pendiente	27
Aprobación del docente-director	28

1. Introducción

1.1. Propósito

Este documento especifica los requisitos funcionales y no funcionales del sistema SGROAS v1.1.0. Sirve como fuente única de verdad para la matriz de trazabilidad, las pruebas automatizadas y la evidencia empírica.

1.2. Alcance

SGROAS es un sistema web para centralizar la gestión de recursos operativos, administrativos y de seguridad de cooperativas de transporte urbano en Ecuador.

Alcance implementado en v1.1.0: autenticación completa (login, refresh, logout con revocación, verificación de correo, recuperación de contraseña), control de acceso por rol (RBAC: ADMIN, COORDINADOR, SEGURIDAD), CRUD completo del módulo funcional (conductores, usuarios, rutas, vehículos, asignaciones, incidentes), subsistema ABD (programaciones, unidades, rutas ABD, incidentes ABD, alertas, catálogos, reportes ABD), caché Redis, procedimientos almacenados y funciones SQL, auditoría mediante triggers, y respaldo con `pg_dump`.

Alcance excluido de este SRS: módulos de facturación y pagos, que no forman parte de la entrega actual y no se especifican ni en este documento ni en la matriz de trazabilidad.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
SGROAS	Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad.
JWT	JSON Web Token (RFC 7519): token firmado que representa afirmaciones sobre un sujeto autenticado.
RBAC	Role-Based Access Control: control de acceso basado en el rol del usuario autenticado.
ORM	Object-Relational Mapping: mapeo objeto-relacional (Spring Data JPA / Hibernate).
SP	Stored Procedure almacenado en el motor de base de datos.
FN	Función SQL almacenada en el motor de base de datos.
MoSCoW	Técnica de priorización: Must, Should, Could, Won't.
shall	Verbo modal normativo de ISO/IEC/IEEE 29148 que indica obligación contractual.
TTL	Time To Live: tiempo de vida de una entrada de caché.

Término	Definición
INCOSE	International Council on Systems Engineering.
CORS	Cross-Origin Resource Sharing.
ABD	subsistema de Administración de Bases de Datos: módulo con tablas propias (programación, unidades, alertas, catálogos) y 28 rutas API bajo <code>/api/abd/**</code> .
Núcleo del sistema	Paquetes excluidos del cómputo de cobertura JaCoCo: <code>ec.edu.uteq.sgroas.abd.*</code> (subsistema ABD completo). La justificación es que el módulo ABD fue desarrollado como práctica independiente de la asignatura de Administración de Bases de Datos y no forma parte del código funcional de la aplicación web. La cobertura del núcleo (sin ABD) es 87,5 % instrucciones; la cobertura global (incluyendo ABD) es 64,8 % instrucciones.

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements Engineering [1].
- INCOSE Guide to Writing Requirements v4 [2].
- Cohn, M. *User Stories Applied* (criterios INVEST) [3].
- Cockburn, A. *Writing Effective Use Cases* [4].
- RFC 7519 — JSON Web Token [5].
- RFC 7807 — Problem Details for HTTP APIs [6].
- OWASP Top 10:2021 [7].
- ISO/IEC 25010:2011 — Calidad del producto software [8].
- Guía de la Entrega Final — PFC Aplicaciones Web 2026–2027, UTEQ.

1.5. Resumen del documento

La sección 2 describe el producto global. La sección 3 contiene los requisitos. La sección 4 resume la trazabilidad. La sección 5 describe el modelo de datos. La sección 6 documenta interfaces. La sección 7 resume lo pendiente. La sección 8 contiene la aprobación del docente.

2. Descripción global

2.1. Perspectiva del producto

SGROAS es un sistema nuevo que reemplaza procesos manuales por una plataforma web centralizada.

2.2. Funciones del producto (resumen)

- Autenticación: login, refresh, logout con revocación, verificación de correo, recuperación de contraseña.
- Control de acceso por rol (RBAC: ADMIN, COORDINADOR, SEGURIDAD).
- CRUD de conductores, usuarios, rutas, vehículos, asignaciones e incidentes (módulo funcional).
- Subsistema ABD: programaciones, unidades, rutas ABD, incidentes ABD, alertas, catálogos, reportes ABD.
- Reportes y agregaciones mediante procedimientos almacenados y funciones SQL.
- Caché Redis en listados paginados con TTL configurable.
- Auditoría automática mediante triggers en la base de datos.
- Respaldo con `pg_dump -format=custom`, frecuencia diaria, retención 30 días.
- Manejo uniforme de errores con ProblemDetails (RFC 7807).

2.3. Características de los usuarios

Rol	Permiso	Descripción
Administrador	Total	Gestiona usuarios, conductores, rutas, vehículos, configuración y subsistema ABD.
Coordinador	Operativo	Gestiona conductores, rutas, vehículos, asignaciones e incidentes.
Seguridad	Consulta	Consulta incidentes, reportes ABD y estado de la flota.

2.4. Restricciones

- El esquema de base de datos se aplica desde migraciones Flyway; queda prohibido `ddl-auto=update`.
- Toda operación no CRUD elemental se implementa como SP o FN en `db/procs/`.
- Queda prohibida la concatenación de entrada de usuario en SQL, JPQL o HQL.
- TLS v1.3 aplica en producción; desarrollo local opera en HTTP plano.

2.5. Supuestos y dependencias

- Docker Desktop y Java 21 disponibles para orquestación y compilación.
- Las credenciales semilla (`admin@sgroas.com` / `admin123`) son exclusivas del entorno de demostración local y no existen en producción. Se documentan en `db/seed.sql` y en el README para facilitar la reproducción del entorno de desarrollo.
- Las mediciones (k6, Lighthouse, JaCoCo) requieren el sistema levantado según el Makefile.

3. Requisitos específicos

3.1. Vocabulario de estados de verificación

Los estados posibles de un requisito son:

Verificado La evidencia citada confirma que el requisito se cumple en el código y puede reproducirse.

Parcialmente verificado El requisito se cumple solo bajo condiciones específicas que se declaran explícitamente (ej: entorno de desarrollo).

Pendiente El requisito fue especificado pero aún no se implementa o no se verificó.

3.2. Vocabulario de métodos de verificación

test Ejecución automatizada de una prueba unitaria, de integración o de carga que confirma el comportamiento del requisito.

demonstration Ejecución manual o semi-automatizada de una operación sobre el sistema en funcionamiento (ej: curl, Postman, navegador) que produce evidencia observable.

analysis Revisión estática del código fuente, configuración o artefactos que confirma la presencia del requisito sin ejecutar el sistema.

inspection Verificación visual de la existencia de un archivo, estructura de paquetes o configuración sin ejecutar código.

3.3. Matriz de permisos por rol y operación

Operación	ADMIN	COORDINADOR	SEGURIDAD
Login / Refresh / Logout	Sí	Sí	Sí
CRUD conductores	Sí	Sí	Solo lectura
CRUD usuarios	Sí	No	No
CRUD rutas	Sí	Sí	Solo lectura
CRUD vehículos	Sí	Sí	Solo lectura
CRUD asignaciones	Sí	Sí	No

Operación	ADMIN	COORDINADOR	SEGURIDAD
CRUD incidentes	Sí	Sí	Sí (lectura+registro)
Reportes funcionales	Sí	Sí	Solo lectura
Subsistema ABD	Sí	No	No
Configuración del sistema	Sí	No	No

3.4. Requisitos funcionales (REQ-F)

Cada requisito sigue el patrón [condición] [sujeto] shall [acción] [objeto] [restricción] de ISO/IEC/IEEE 29148, con identificador persistente, rationale, prioridad MoSCoW, criterio de aceptación medible y método de verificación, cumpliendo las características INCOSE C1–C9.

3.4.1. REQ-F-001 — Inicio de sesión

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** El acceso al sistema requiere autenticación previa.
- **Enunciado:** El sistema deberá autenticar al usuario mediante `POST /api/auth/login` y, tras validar credenciales, emitir un JWT firmado y establecer una `cookie` `HttpOnly` + `Secure` + `SameSite=Strict` (RFC 7519).
- **Criterio de aceptación:** Con credenciales válidas responde `200 OK` y `cookie` con el JWT; con credenciales inválidas responde `401 Unauthorized` con `ProblemDetails`.
- **Verificación:** test (`AuthServiceTest`) y demonstration (Postman).
- **Trazabilidad:** HU-001, CU-001, `AuthController`, `POST /api/auth/login`. **Estado:** verificado.

3.4.2. REQ-F-002 — Consulta de usuario autenticado

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** El frontend necesita obtener el perfil del usuario logueado.
- **Enunciado:** El sistema deberá retornar los datos del usuario autenticado mediante `GET /api/auth/me`, extrayendo la identidad del JWT de la `cookie`.
- **Criterio de aceptación:** Con JWT válido responde `200 OK` con nombre, correo y rol; sin JWT responde `401`.

- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, GET /api/auth/me. **Estado:** verificado.

3.4.3. REQ-F-003 — Renovación de token

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** Evitar reautenticación frecuente manteniendo la seguridad.
- **Enunciado:** El sistema deberá renovar el JWT mediante POST /api/auth/refresh verificando el *refresh token*.
- **Criterio de aceptación:** Refresh token válido responde 200 OK con nuevo JWT; inválido o revocado responde 401.
- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, POST /api/auth/refresh. **Estado:** verificado.

3.4.4. REQ-F-004 — Cierre de sesión con revocación

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** Permitir revocar la sesión activa.
- **Enunciado:** El sistema deberá revocar el token (JTI en lista negra Redis con TTL igual a su tiempo restante de expiración) mediante POST /api/auth/logout.
- **Criterio de aceptación:** Responde 204 No Content y el token queda revocado en Redis.
- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, POST /api/auth/logout. **Estado:** verificado.

3.4.5. REQ-F-005 — Verificación de correo electrónico

- **Tipo:** Funcional. **Prioridad:** Should.
- **Rationale:** Confirmar la titularidad del correo antes de permitir el acceso completo.
- **Enunciado:** El sistema deberá verificar el correo electrónico del usuario mediante POST /api/auth/verify-email con un código numérico de 6 dígitos.

- **Criterio de aceptación:** Código válido responde 200 OK; código inválido o expirado responde 400 Bad Request.
- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, POST /api/auth/verify-email. **Estado:** verificado.

3.4.6. REQ-F-006 — Reenvío de código de verificación

- **Tipo:** Funcional. **Prioridad:** Should.
- **Rationale:** Permitir reenviar el código si el usuario no lo recibió.
- **Enunciado:** El sistema deberá reenviar el código de verificación mediante POST /api/auth/resent-code, aplicando un límite de reenvíos por ventana de tiempo.
- **Criterio de aceptación:** Responde 200 OK con confirmación; exceso de reenvíos responde 429 Too Many Requests.
- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, POST /api/auth/resent-code. **Estado:** verificado.

3.4.7. REQ-F-007 — Solicitud de restablecimiento de contraseña

- **Tipo:** Funcional. **Prioridad:** Should.
- **Rationale:** Permitir recuperar el acceso cuando el usuario olvidó su contraseña.
- **Enunciado:** El sistema deberá enviar un enlace de restablecimiento al correo registrado mediante POST /api/auth/forgot-password. El enlace expirará en 24 horas.
- **Criterio de aceptación:** Responde 200 OK independientemente de si el correo existe (para no revelar información). El enlace generado es válido por 24 horas.
- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, POST /api/auth/forgot-password. **Estado:** verificado.

3.4.8. REQ-F-008 — Restablecimiento efectivo de contraseña

- **Tipo:** Funcional. **Prioridad:** Should.
- **Rationale:** Completar el flujo de recuperación de contraseña.
- **Enunciado:** El sistema deberá cambiar la contraseña del usuario mediante `POST /api/auth/reset-password` validando el token del enlace. Las sesiones activas del usuario se revocan al cambiar la contraseña.
- **Criterio de aceptación:** Token válido y contraseña nueva cumple política responde 200 OK; token inválido o expirado responde 400 Bad Request.
- **Verificación:** test (AuthServiceTest).
- **Trazabilidad:** HU-001, AuthController, `POST /api/auth/reset-password`. **Estado:** verificado.

3.4.9. REQ-F-009 — Control de acceso por rol (RBAC)

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** Restringir acceso según el rol del usuario autenticado.
- **Enunciado:** El sistema deberá restringir el acceso a cada endpoint protegido según el rol del usuario autenticado, rechazando con 403 Forbidden cualquier solicitud de un rol no autorizado. La configuración utiliza `@EnableMethodSecurity` y anotaciones `@PreAuthorize` en los controladores.
- **Criterio de aceptación:** Un usuario con rol no autorizado para un recurso recibe 403 Forbidden.
- **Verificación:** test (SecurityConfigTest) y demonstration (Postman).
- **Trazabilidad:** HU-002, CU-002, SecurityConfig, `@PreAuthorize` en controladores. **Estado:** verificado.

3.4.10. REQ-F-010 — Listado de conductores

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** La gestión de la flota requiere el listado de conductores.
- **Enunciado:** El sistema deberá retornar un listado paginado y filtrable de conductores mediante `GET /api/conductores`.
- **Criterio de aceptación:** Responde 200 OK con página, tamaño y total. La caché Redis almacena el resultado con TTL configurable.

- **Verificación:** test (ConductorServiceTest) y demonstration (Postman).
- **Trazabilidad:** HU-002, CU-002, ConductorController. **Estado:** verificado.

3.4.11. REQ-F-011 — Consulta de conductor por id

- **Tipo:** Funcional. **Prioridad:** Must.
- **Enunciado:** El sistema deberá retornar un conductor por su identificador mediante GET /api/conductores/{id}.
- **Criterio de aceptación:** Responde 200 OK con el objeto o 404 Not Found.
- **Verificación:** test (ConductorServiceTest). **Estado:** verificado.

3.4.12. REQ-F-012 — Alta de conductor

- **Tipo:** Funcional. **Prioridad:** Must.
- **Enunciado:** El sistema deberá crear un conductor mediante POST /api/conductores validando unicidad de cédula.
- **Criterio de aceptación:** Responde 201 Created; cédula duplicada responde 409 Conflict.
- **Verificación:** test (ConductorServiceTest). **Estado:** verificado.

3.4.13. REQ-F-013 — Actualización de conductor

- **Tipo:** Funcional. **Prioridad:** Must.
- **Enunciado:** El sistema deberá actualizar un conductor mediante PUT /api/conductores/{id}.
- **Criterio de aceptación:** Responde 200 OK con el objeto actualizado.
- **Verificación:** test (ConductorServiceTest). **Estado:** verificado.

3.4.14. REQ-F-014 — Eliminación lógica de conductor

- **Tipo:** Funcional. **Prioridad:** Must.
- **Enunciado:** El sistema deberá desactivar un conductor mediante DELETE /api/conductores/{id} sin borrar la fila.
- **Criterio de aceptación:** Responde 204 No Content; el estado cambia a inactivo.
- **Verificación:** test (ConductorServiceTest). **Estado:** verificado.

3.4.15. REQ-F-015 a REQ-F-019 — CRUD de usuarios

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** El administrador debe gestionar las cuentas del sistema.
- **Enunciados:**
 - REQ-F-015 Listado paginado: GET /api/usuarios. **Estado:** verificado.
 - REQ-F-016 Consulta por id: GET /api/usuarios/{id}. **Estado:** verificado.
 - REQ-F-017 Alta: POST /api/usuarios. **Estado:** verificado.
 - REQ-F-018 Actualización: PUT /api/usuarios/{id}. **Estado:** verificado.
 - REQ-F-019 Eliminación lógica: DELETE /api/usuarios/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; el rol se restringe a ADMIN, COORDINADOR, SEGURIDAD.
- **Verificación:** test (UsuarioServiceTest) y demonstration.
- **Trazabilidad:** HU-003, CU-003, UsuarioController.

3.4.16. REQ-F-020 a REQ-F-024 — CRUD de rutas

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Las rutas son la unidad base de planificación operativa.
- **Enunciados:**
 - REQ-F-020 Listado paginado: GET /api/rutas. **Estado:** verificado.
 - REQ-F-021 Consulta por id: GET /api/rutas/{id}. **Estado:** verificado.
 - REQ-F-022 Alta: POST /api/rutas. **Estado:** verificado.
 - REQ-F-023 Actualización: PUT /api/rutas/{id}. **Estado:** verificado.
 - REQ-F-024 Eliminación lógica: DELETE /api/rutas/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; nombre de ruta único; distancia mayor a cero.
- **Verificación:** test (RutaServiceTest).
- **Trazabilidad:** HU-004, CU-004, RutaController.

3.4.17. REQ-F-025 a REQ-F-029 — CRUD de vehículos

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Inventario de la flota vehicular.
- **Enunciados:**
 - REQ-F-025 Listado paginado: GET /api/vehiculos. **Estado:** verificado.
 - REQ-F-026 Consulta por id: GET /api/vehiculos/{id}. **Estado:** verificado.
 - REQ-F-027 Alta: POST /api/vehiculos. **Estado:** verificado.
 - REQ-F-028 Actualización: PUT /api/vehiculos/{id}. **Estado:** verificado.
 - REQ-F-029 Desactivación lógica: DELETE /api/vehiculos/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; placa única; año en rango válido.
- **Verificación:** test (VehiculoServiceTest).
- **Trazabilidad:** HU-005, CU-005, VehiculoController.

3.4.18. REQ-F-030 a REQ-F-034 — CRUD de asignaciones

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Asignar conductores y vehículos a rutas específicas.
- **Enunciados:**
 - REQ-F-030 Listado paginado: GET /api/asignaciones. **Estado:** pendiente — endpoint devuelve 500 en producción por LazyInitializationException.
 - REQ-F-031 Consulta por id: GET /api/asignaciones/{id}. **Estado:** verificado.
 - REQ-F-032 Alta: POST /api/asignaciones. **Estado:** verificado.
 - REQ-F-033 Actualización: PUT /api/asignaciones/{id}. **Estado:** verificado.
 - REQ-F-034 Eliminación: DELETE /api/asignaciones/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; validación de disponibilidad del conductor y vehículo para la fecha.
- **Verificación:** test (AsignacionRutaServiceTest).
- **Trazabilidad:** HU-006, CU-006, AsignacionRutaController.

3.4.19. REQ-F-035 a REQ-F-039 — CRUD de incidentes

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Registro estructurado de incidentes para análisis de seguridad.
- **Enunciados:**
 - REQ-F-035 Listado paginado: GET /api/incidentes. **Estado:** verificado.
 - REQ-F-036 Consulta por id: GET /api/incidentes/{id}. **Estado:** verificado.
 - REQ-F-037 Alta: POST /api/incidentes. **Estado:** verificado.
 - REQ-F-038 Actualización: PUT /api/incidentes/{id}. **Estado:** verificado.
 - REQ-F-039 Eliminación lógica: DELETE /api/incidentes/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; gravedad restringida a ALTA, MEDIA, BAJA; fecha no futura.
- **Verificación:** test (IncidenteServiceTest).
- **Trazabilidad:** HU-007, CU-006, IncidenteController.

3.4.20. REQ-F-040 a REQ-F-045 — Reportes funcionales

- **Tipo:** Funcional. **Prioridad:** Should (cada uno).
- **Rationale:** Generación de reportes operativos y estadísticas para toma de decisiones.
- **Enunciados:**
 - REQ-F-040 Reporte de rendimiento de rutas: GET /api/reportes/rendimiento-rutas (SP sp_reporte_rendimiento_rutas). **Estado:** verificado.
 - REQ-F-041 Estadísticas generales: GET /api/reportes/estadisticas-generales (FN fn_estadisticas_generales). **Estado:** verificado.
 - REQ-F-042 Licencias por vencer: GET /api/reportes/licencias-por-vencer (FN fn_licencias_por_vencer). **Estado:** verificado.
 - REQ-F-043 Incidentes por gravedad: GET /api/reportes/incidentes-por-gravedad (SP sp_incidentes_por_gravedad). **Estado:** verificado.
 - REQ-F-044 Incidentes por rango: GET /api/reportes/incidentes-por-rango (SP sp_obtener_incidentes_por_rango). **Estado:** verificado.
 - REQ-F-045 Asignaciones activas por conductor: GET /api/reportes/asignaciones-activas- (SP sp_asignaciones_activas_por_conductor). **Estado:** verificado.

- **Criterio de aceptación:** Cada reporte retorna los campos esperados; los SP no construyen SQL dinámico.
- **Verificación:** test (ReporteControllerTest) y demonstration (Postman).
- **Trazabilidad:** HU-008, CU-006, ReportesController.

3.4.21. REQ-F-046 a REQ-F-050 — CRUD de programaciones (ABD)

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** El subsistema ABD gestiona programaciones diarias de la flota.
- **Enunciados:**
 - REQ-F-046 Listado paginado: GET /api/abd/programaciones. **Estado:** verificado.
 - REQ-F-047 Alta: POST /api/abd/programaciones. **Estado:** verificado.
 - REQ-F-048 Actualización: PUT /api/abd/programaciones/{id}. **Estado:** verificado.
 - REQ-F-049 Eliminación: DELETE /api/abd/programaciones/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; validación de existencia de ruta, unidad y conductor.
- **Verificación:** test y demonstration.
- **Trazabilidad:** ProgramacionAbdController.

3.4.22. REQ-F-050 a REQ-F-054 — CRUD de unidades (ABD)

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Inventario de unidades vehiculares del esquema ABD.
- **Enunciados:**
 - REQ-F-050 Listado paginado: GET /api/abd/unidades. **Estado:** verificado.
 - REQ-F-051 Consulta por id: GET /api/abd/unidades/{id}. **Estado:** verificado.
 - REQ-F-052 Alta: POST /api/abd/unidades. **Estado:** verificado.
 - REQ-F-053 Actualización: PUT /api/abd/unidades/{id}. **Estado:** verificado.
 - REQ-F-054 Eliminación: DELETE /api/abd/unidades/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; placa única; estado válido.

- **Verificación:** test y demonstration.
- **Trazabilidad:** UnidadAbdController.

3.4.23. REQ-F-055 a REQ-F-059 — CRUD de rutas ABD

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Rutas del esquema ABD entre terminales.
- **Enunciados:**

REQ-F-055 Listado paginado: GET /api/abd/rutas. **Estado:** verificado.
REQ-F-056 Consulta por id: GET /api/abd/rutas/{id}. **Estado:** verificado.
REQ-F-057 Alta: POST /api/abd/rutas. **Estado:** verificado.
REQ-F-058 Actualización: PUT /api/abd/rutas/{id}. **Estado:** verificado.
REQ-F-059 Eliminación: DELETE /api/abd/rutas/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; terminal origen distinto de destino.
- **Verificación:** test y demonstration.
- **Trazabilidad:** RutaAbdController.

3.4.24. REQ-F-060 a REQ-F-063 — CRUD de incidentes ABD

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** Incidentes del esquema ABD para análisis de seguridad.
- **Enunciados:**

REQ-F-060 Listado paginado: GET /api/abd/incidentes. **Estado:** verificado.
REQ-F-061 Alta: POST /api/abd/incidentes. **Estado:** verificado.
REQ-F-062 Actualización: PUT /api/abd/incidentes/{id}. **Estado:** verificado.
REQ-F-063 Eliminación: DELETE /api/abd/incidentes/{id}. **Estado:** verificado.
- **Criterio de aceptación:** Respuestas REST correctas; nivel_{usuario} restringido a ALTO, MEDIO o BAJO.
- **Verificación:** test y demonstration.
- **Trazabilidad:** IncidenteAbdController.

3.4.25. REQ-F-064 — Consulta de alertas ABD

- **Tipo:** Funcional. **Prioridad:** Should.
- **Enunciado:** El sistema deberá retornar las alertas generadas por incidentes de nivel ALTO mediante `GET /api/abd/alertas`.
- **Criterio de aceptación:** Responde 200 OK con listado de alertas.
- **Verificación:** demonstration. **Estado:** verificado.
- **Trazabilidad:** AlertaAbdController.

3.4.26. REQ-F-065 — Consulta de catálogos ABD

- **Tipo:** Funcional. **Prioridad:** Should.
- **Enunciado:** El sistema deberá retornar los catálogos del esquema ABD (provincias, ciudades, terminales, roles) mediante `GET /api/abd/catalogos`.
- **Criterio de aceptación:** Responde 200 OK con las cuatro colecciones.
- **Verificación:** demonstration. **Estado:** verificado.
- **Trazabilidad:** CatalogoAbdController.

3.4.27. REQ-F-066 — Consulta de conductores ABD

- **Tipo:** Funcional. **Prioridad:** Should.
- **Enunciado:** El sistema deberá retornar el listado de conductores del esquema ABD mediante `GET /api/abd/conductores`.
- **Criterio de aceptación:** Responde 200 OK con listado paginado.
- **Verificación:** demonstration. **Estado:** verificado.
- **Trazabilidad:** ConductorAbdController.

3.4.28. REQ-F-067 a REQ-F-071 — Reportes ABD

- **Tipo:** Funcional. **Prioridad:** Should (cada uno).
- **Rationale:** Reportes analíticos del subsistema ABD.
- **Enunciados:**
 - REQ-F-067** Resumen general: `GET /api/abd/reportes/resumen`. **Estado:** verificado.

REQ-F-068 Incidentes por nivel: GET /api/abd/reportes/incidentes-por-nivel.
Estado: verificado.

REQ-F-069 Incidentes por estado: GET /api/abd/reportes/incidentes-por-estado.
Estado: verificado.

REQ-F-070 Unidades por estado: GET /api/abd/reportes/unidades-por-estado.
Estado: verificado.

REQ-F-071 Programaciones por estado: GET /api/abd/reportes/programaciones-por-estado.
Estado: verificado.

REQ-F-072 Programaciones por mes: GET /api/abd/reportes/programaciones-por-mes.
Estado: verificado.

REQ-F-073 Top rutas: GET /api/abd/reportes/top-rutas. **Estado:** verificado.

- **Criterio de aceptación:** Cada reporte retorna los campos esperados.
- **Verificación:** demonstration (Postman).
- **Trazabilidad:** ReporteAbdController.

3.4.29. REQ-F-074 — Política de contraseñas

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** El flujo de restablecimiento requiere una política de contraseñas que las contraseñas deben cumplir.
- **Enunciado:** El sistema deberá exigir que las contraseñas tengan mínimo 8 caracteres, incluyendo al menos una mayúscula, una minúscula y un número. Las contraseñas se almacenan con hash BCrypt.
- **Criterio de aceptación:** Contraseña que no cumpla la política es rechazada con 400 Bad Request. Contraseña válida se almacena hashada.
- **Verificación:** test. **Estado:** verificado.
- **Trazabilidad:** AuthController.

3.4.30. REQ-F-075 — Protección de datos personales

- **Tipo:** Funcional. **Prioridad:** Should.
- **Rationale:** El sistema almacena cédula, licencia de conductores e incidentes con datos personales identificables.

- **Enunciado:** El sistema deberá permitir la eliminación lógica de registros de conductores y incidentes que contengan datos personales, conservando los datos de forma agregada para reportes.
- **Criterio de aceptación:** La eliminación lógica cambia el estado sin borrar la fila; los reportes muestran datos agregados sin exponer personas individuales.
- **Verificación:** inspection. **Estado:** verificado.
- **Trazabilidad:** ConductorController, IncidenteController.

3.4.31. REQ-F-076 — Respaldo de la base de datos

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** Garantizar la recuperabilidad de datos ante fallos.
- **Enunciado:** El sistema deberá disponer de un respaldo diario de la base de datos mediante `pg_dump -format=custom` con retención de 30 días, ejecutado por el script `scripts/backup-prod.sh`.
- **Criterio de aceptación:** El archivo de respaldo es verificable con `pg_restore -list` sin errores.
- **Verificación:** demonstration (`backups/sgroas-2026-08-28.sql.dump`, 17,25 MB, 228 entradas TOC). **Estado:** verificado.
- **Trazabilidad:** `scripts/backup-prod.sh`, `docs/despliegue/BACKUP.md`.

3.4.32. REQ-F-077 — Accesibilidad y usabilidad

- **Tipo:** Funcional. **Prioridad:** Should.
- **Rationale:** La aplicación debe ser usable y accesible para los usuarios del sistema.
- **Enunciado:** El frontend Angular 20 deberá alcanzar un puntaje mínimo de 85 en Lighthouse Accessibility (desktop) y una satisfacción del usuario (SUS) media de al menos 68 sobre 100.
- **Criterio de aceptación:** Lighthouse Accessibility ≥ 85 (desktop); SUS media ≥ 68 ($n \geq 15$).
- **Verificación:** test (Lighthouse, SUS). **Estado:** verificado (Lighthouse A=91 desktop, SUS=68,5 con $n=15$).
- **Trazabilidad:** `docs/mediciones/lighthouse/`, `docs/mediciones/sus/`.

4. Requisitos no funcionales (REQ-NF)

4.0.1. REQ-NF-001 — Cabeceras de seguridad HTTP

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Seguridad (OWASP A05).
- **Enunciado:** Toda respuesta HTTP incluirá Strict-Transport-Security, X-Frame-Options: DENY, X-Content-Type-Options: nosniff y Content-Security-Policy.
- **Criterio de aceptación:** curl -I contra la raíz muestra las cuatro cabeceras.
- **Verificación:** test y demonstration (docs/mediciones/sec/). **Estado:** verificado.

4.0.2. REQ-NF-002 — Cifrado en tránsito

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Seguridad (OWASP A02).
- **Enunciado:** La conexión utilizará TLS v1.3 con suites AEAD en producción. Desarrollo local opera en HTTP plano.
- **Criterio de aceptación:** En producción, curl -v muestra TLSv1.3.
- **Verificación:** demonstration. **Estado:** parcialmente verificado — validado únicamente en desarrollo local (HTTP plano); producción requiere configuración de reverse proxy con TLS.

4.0.3. REQ-NF-003 — Rendimiento del listado

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Rendimiento.
- **Enunciado:** El GET /api/conductores con caché caliente responderá con p95 menor a 200 ms.
- **Criterio de aceptación:** Tres corridas k6 (50 VUs, 30 s) con p95 medio de 23,01 ms y 0 % de errores.
- **Verificación:** test (k6), reporte docs/mediciones/perf/. **Estado:** verificado.

4.0.4. REQ-NF-004 — Protección contra inyección SQL

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Seguridad (OWASP A03).
- **Enunciado:** Los payloads de entrada se parametrizan estrictamente; los SP no construyen SQL dinámico.
- **Criterio de aceptación:** El auditor `scripts/audit-sql-dynamic.sh` no reporta violaciones.
- **Verificación:** test y analysis (`docs/mediciones/sec/`). **Estado:** verificado.

4.0.5. REQ-NF-005 — Limitación de intentos de login

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Seguridad (OWASP A07).
- **Enunciado:** Seis intentos fallidos consecutivos desde la misma IP responderán 429 Too Many Requests.
- **Criterio de aceptación:** La séptima petición responde 429.
- **Verificación:** demonstration (`docs/mediciones/sec/`). **Estado:** verificado.

4.0.6. REQ-NF-006 — Cobertura de pruebas

- **Tipo:** No funcional. **Prioridad:** Should.
- **Categoría:** Mantenibilidad.
- **Enunciado:** El reporte JaCoCo sobre el núcleo del sistema (definido en la sección 1.3, excluyendo el paquete `ec.edu.uteq.sgroas.abd.*`) alcanzará al menos 70 % de instrucciones.
- **Criterio de aceptación:** `jacoco.csv` del núcleo con cobertura ≥ 70 % instrucciones.
- **Verificación:** test (`docs/mediciones/jacoco/`). **Estado:** verificado — núcleo: 87,5 % instrucciones, 87,9 % ramas, 95,5 % líneas, 222 pruebas. Sistema completo: 64,8 % instrucciones, 46,2 % ramas, 73,2 % líneas.

4.0.7. REQ-NF-007 — Manejo uniforme de errores

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Usabilidad.
- **Enunciado:** Los errores seguirán el formato ProblemDetails (RFC 7807) con type, title, status, detail e instance.
- **Verificación:** test y demonstration (Postman). **Estado:** verificado.

4.0.8. REQ-NF-008 — Arquitectura en capas

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Mantenibilidad.
- **Enunciado:** La arquitectura del backend se organizará en capas separadas de presentación (controller), lógica de negocio (service) y acceso a datos (repository/entity).
- **Verificación:** inspection (estructura de paquetes). **Estado:** verificado.

4.0.9. REQ-NF-009 — Estrategia de acceso a datos

- **Tipo:** No funcional. **Prioridad:** Must.
- **Categoría:** Mantenibilidad / Seguridad.
- **Enunciado:** Toda operación de BD que no sea un CRUD elemental se implementará como SP o FN versionado en db/procs/, quedando prohibida la concatenación de entrada de usuario en JPQL, HQL o SQL nativo.
- **Verificación:** test, scripts/audit-sql-dynamic.sh, docs/basedatos/CATALOGO-SP.md. **Estado:** verificado — 6 SP + 5 FN = 11 objetos en db/procs/, 0 hallazgos de SQL dinámico.

4.0.10. REQ-NF-010 — Disponibilidad de Redis

- **Tipo:** No funcional. **Prioridad:** Should.
- **Categoría:** Disponibilidad.
- **Enunciado:** Ante la indisponibilidad de Redis, el sistema deberá seguir operando sin caché (respuestas sin caché) y rechazando las solicitudes de logout (revocación de tokens) con un error controlado 503 **Service Unavailable**.

- **Criterio de aceptación:** El listado de conductores retorna datos sin caché; el logout retorna 503 cuando Redis no está disponible.
- **Verificación:** inspection. **Estado:** verificado.
- **Trazabilidad:** SecurityConfig, ConductorController.

5. Trazabilidad (resumen)

La matriz end-to-end se versiona en `docs/trazabilidad/matriz.csv`. Contiene 77 requisitos (69 funcionales REQ-F-001 a REQ-F-077 + 8 no funcionales REQ-NF-001 a REQ-NF-010, ajustado tras correcciones de revisión). La validez se comprueba con `scripts/validate-tracea` que verifica coherencia entre el SRS y la matriz.

5.1. Trazabilidad histórica

Las modificaciones a los requisitos se registran en `docs/requisitos/CHANGELOG-REQ.md` siguiendo Keep a Changelog; las decisiones de diseño se documentan como ADR en `docs/adr/`.

6. Modelo de datos (referencia)

El modelo relacional se define en las migraciones Flyway (V1–V13). Las entidades principales del módulo funcional son: Usuario, Conductor, Vehiculo, Ruta, AsignacionRuta, Incidente. El subsistema ABD añade: Provincia, Ciudad, Terminal, Rol, Unidad, Programacion, Alerta, Auditoría. El DER completo se encuentra en `docs/arquitectura/`.

7. Interfaces de usuario (referencia)

El frontend Angular 20 implementa pantallas para: login, dashboard, conductores, usuarios, rutas, vehículos, asignaciones, incidentes, y los módulos ABD (programaciones, unidades, alertas, reportes ABD). El diseño responsivo se verifica con Lighthouse (desktop P=95, A=91, BP=92, SEO=90; móvil P=77, A=91, BP=92, SEO=90).

8. Observaciones e información pendiente

Resuelto @EnableMethodSecurity: la anotación está presente en SecurityConfig.java línea 26; ReporteController tiene ocho anotaciones @PreAuthorize efectivas.

Resuelto Cookie JWT bit Secure: el cookie se construye con `.secure(cookieSecure)`, configurable por entorno mediante `app.cookie.secure`.

Pendiente GET /api/asignaciones: devuelve 500 en producción por LazyInitializationException (REQ-F-030 marcado pendiente).

APROBACIÓN DEL DOCENTE-DIRECTOR

Proyecto: SGROAS — Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad para Cooperativas de Transporte.

Documento: Especificación de Requisitos de Software (SRS) — v1.1.0.

Fecha de emisión: 09 de septiembre de 2026.

Estudiantes:

- Castro Espinoza Kevin Moisés (ORCID: 0009-0004-4935-0439)
- Escudero Plaza María del Rosario (ORCID: 0009-0007-3212-9924)
- Tejada Bajaña Luis Alejandro (ORCID: 0009-0006-2080-1798)

El docente-director declara:

Firma del docente-director:

Fecha de aprobación:

Ing. Guerrero Ulloa Gleiston Cicerón

Docente-director

Observaciones del docente:

Referencias

- [1] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, “ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering,” ISO/IEC/IEEE, Technical Report, 2018.
- [2] International Council on Systems Engineering, “INCOSE Guide to Writing Requirements,” INCOSE — Requirements Working Group, Technical Report INCOSE-TP-2010-006-04, 2023.
- [3] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA: Addison-Wesley Professional, 2004.
- [4] A. Cockburn, *Writing Effective Use Cases*. Boston, MA: Addison-Wesley Professional, 2001.
- [5] M. Jones, J. Bradley y N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [6] M. Nottingham y E. Wilde, *Problem Details for HTTP APIs*, RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [7] OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, 2021.
- [8] ISO/IEC, *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE)*, 2011.